

S.Y.B.SC.(DATA SCIENCE) SEM- III

Subject: Data Structure – I

Unit1 : Introduction to Data Structures and Algorithm Analysis

By,

Mrs. Reshma Measurekar

Dr. D. Y. Patil ACS College,

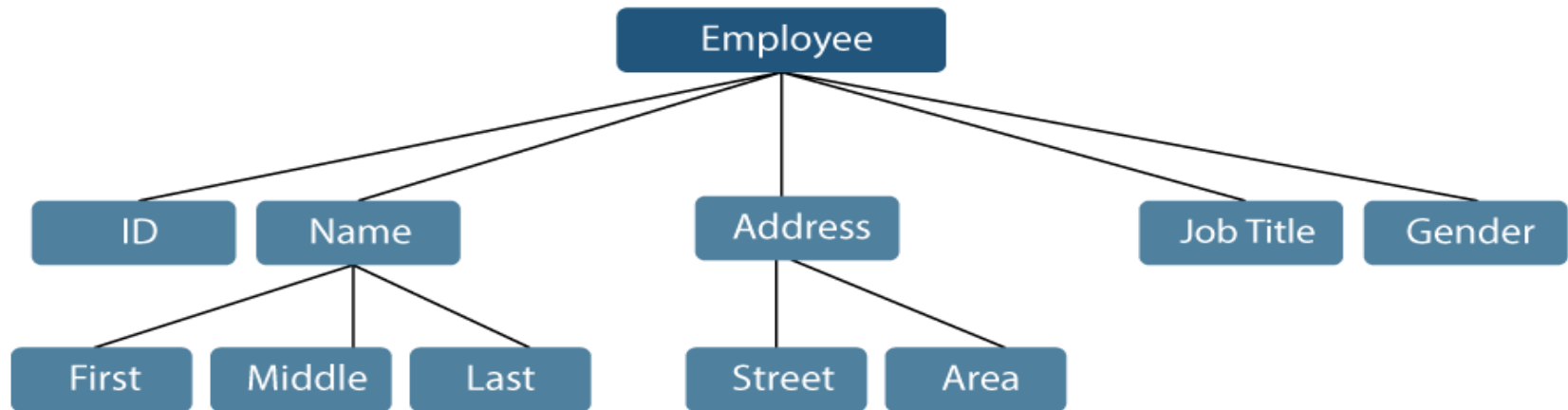
Pimpri, Pune 18.

Unit1 : Introduction to Data Structures and Algorithm Analysis

- Overview of Data Structure
 - Need of Data Structure,
 - Definitions: Data and information, Data type, Data object, ADT, Data Structure
 - Types of Data Structures: Linear vs NonLinear, Static vs. Dynamic
- Complexity Analysis
 - Space and time complexity
 - Asymptotic notations (Big O, Omega Ω , Theta Θ),
 - Best, Worst, Average case analysis

Data

- **Data:** *Raw, unprocessed facts, figures, or symbols that can be processed and analyzed to extract useful information.* Data can be numerical, textual, or even visual.
- **Data** is a Piece of Information.
- **Example:** Employee name can be broken down into three sub-items: First, Middle, and Last. However, an ID assigned to an employee will generally be considered a single item.



Categories of Data

- Data can be categorized into two groups:
 - **Structured data:** Structured data is organized and easier to work with.
 - **Unstructured data:** Unstructured data is not organized.

Structured data



Unstructured data



Types of Data

- Data is divided into atomic and non-atomic data:

1. Atomic data:

It is *single or non-decomposable entity*. Data cannot be broken into subfields.

Ex.: EmployeeId

2. Non-atomic data:

Data can be *broken into subfields*.

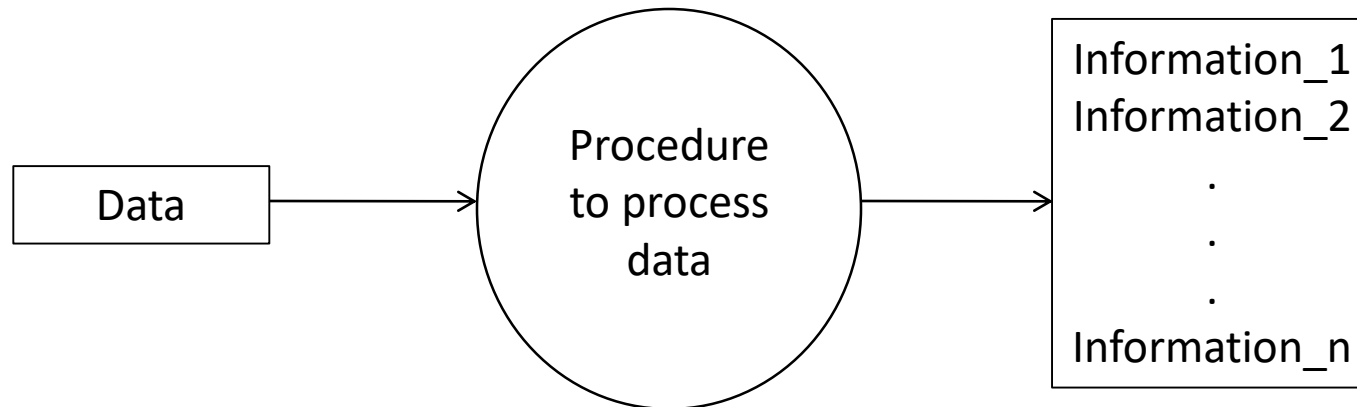
It is also called as structured data or Composite data.

Ex.: EmployeeName

EmployeeName is divided into Fname, Mname, Lname

Information

- **Information:** *Data that has been processed, organized, and presented in a meaningful way to provide context and allow for informed decision-making.*
- Information is defined as **meaningful, logical and processed data.**
- Data are raw facts from which the required information is produced.



Data Types

- Data type is used to describe the type of data that variable may hold in the programming language.
- Data type=Permitted data values + operations
- Data type is also defined as set of values and set of operations that operate on those values.

Data object

- A data object represents a *container for data values – a place where data values are stored and later retrieved*. OR
- *A storage location containing a value or group of values that can be accessed using an identifier or a complex expression.*
- Data objects refers to the set of all elements say D, which may be finite or infinite.
- Ex.: The data object of “alphabet” is defined as :
$$D=\{A, B, \dots Z\}$$
- Ex.: The data object of “integer” is defined as :
$$D=\{\dots -2, -1, 0, 1, 2, \dots\}$$
- Data object is binded with type, location, value, name & component.

ADT

- It is a concept or model of a data type that defines its behavior and operations independently of how it is implemented.
- Abstract Data type (ADT) specifies a set of data and set of operations that can be operated on that data.
- ADTs focus on **what** operations are to be performed but **not how** these operations will be implemented.
- i.e it does **not** specify how data will be organized in memory and what algorithms will be used for implementing the operations.
- It is called “abstract” because it gives an implementation-independent view.
- ADT acts as blackbox.
- Ex. :List ADT, Stack ADT, Queue ADT

Advantages of ADT

- **Encapsulation:** It ensures that data cannot be corrupted.
- **Reusability:** ADT is reusable and ensures robust data structure.
- It **reduces coding efforts**.
- **Hiding Implementation** details.
- ADT is useful guidelines to implement and a useful tool for programmers who wish to use the data type correctly.

Introduction to Data Structure

- It is a programming construct to implement an ADT.
- It is the physical implementation of ADT.
- A DS that is implementing as ADT contains collection of variables for storing the data specified in ADT and the algorithms for implementing the operations specified in ADT.

Data Structure

- *Data structure is a way of arranging data on a computer so that it can be accessed and updated efficiently.*
- Data Structure is used for organizing the data in memory.
- There are various ways of organizing the data in the memory, like array, list, stack, queue, and many more.
- The data structure isn't a programming language like C, C++, Java, etc. It is a set of algorithms that can be used in any programming language to organize the data in the memory.

Data Structure

- Data structure is a way of collecting and organizing data in such a way that operations can be performed on data in an efficient way.
- Logical or mathematical model of a particular organization of data in main memory is called as a data structure.
- **Formal Definition:** A data structure is defined as, a set of domains D , a designated domain ϵD (belongs to D), a set of functions F , and a set of axioms A . The triple (D, F, A) denotes the data structure d and it will usually be written as d , where,
 - **Domain(D):** Denotes data object.
 - **Function(F):** Denotes the set of operations that can be performed on the data objects
 - **Axioms(A):** Denotes the properties and rules of the operation.

Need of Data Structure

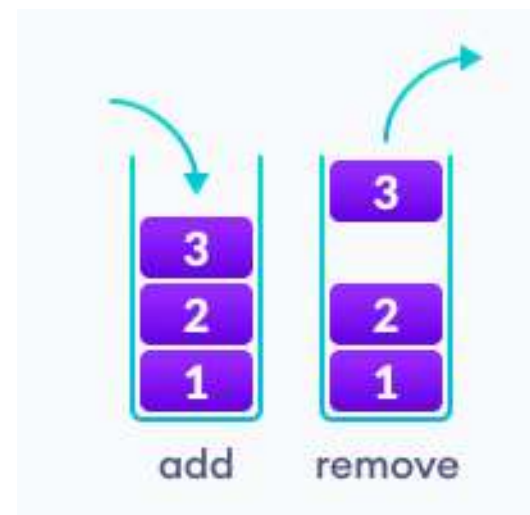
- Data structure modification is easy.
- It requires less time.
- Save storage memory space.
- Data representation is easy.
- Easy access to the large database.
- As applications are becoming more complex and the amount of data is increasing day by day, which may cause problems with processing speed, searching data, handling multiple requests etc.
- Data structure provides a way of organizing, managing, and storing data efficiently. With the help of data structure, the data items can be traversed easily. Data structure provides efficiency, reusability and abstraction. It plays an important role in enhancing the performance of a program because the main function of the program is to store and retrieve the user's data as fast as possible.

Operations on Data Structure

- Some common operations performed on Data Structure
 - Traversing
 - Searching
 - Insertion
 - Deletion
 - Sorting
 - Merging

Linear data structure

- Data structure is said to be linear if data elements are arranged sequentially.
- Each element is attached to its previous and next adjacent elements.
- *Examples: array, stack, queue, linked list, etc.*



Non-Linear data structure

- Data structure is said to be Non-linear if its elements form a *hierarchical classification* where data elements are appears at various level.
- All the elements can't be traversed in a single run only.
- Example: Tree, Graph

Static and Dynamic data structure

- **Static data structure:** Static data structure has a *fixed memory size*. It is *easier to access* the elements in a static data structure.

Example: array

- **Dynamic data structure:** In the dynamic data structure, the *size is not fixed*. It can be *randomly updated during the runtime* which may be considered efficient concerning the memory (space) complexity of the code.

Example: linked list

Linear Data Structures	Non Linear Data Structures
The data items are arranged in sequential order, one after the other.	The data items are arranged in non-sequential order (hierarchical manner).
All the items are present on the single layer.	The data items are present at different layers.
It can be traversed on a single run.	It requires multiple runs.
The memory utilization is not efficient.	Different structures utilize memory in different efficient ways depending on the need.
The time complexity increase with the data size.	Time complexity remains the same.
Example: Arrays, Stack, Queue	Example: Tree, Graph, Map